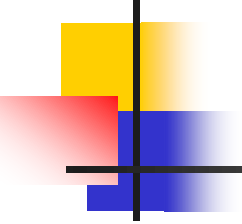


Evolutionary Requirements

and

Use cases vs User stories



Selvundersøgelse

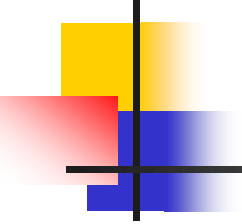
I grupper af 4 studerende: (**I har 30 min**)

Find information om disse emner-

- Requirements / krav til softwareudviklingsprojekter
- Use Cases
- User Stories
- Use Cases vs User Stories

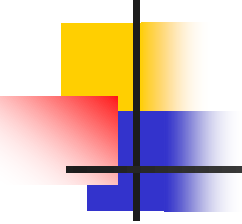
Skriv det i finder ned og skab en fælles forståelse for emnerne

Udarbejd spørgsmål til emnerne som kan stilles under gennemgangen af underviseren



Requirements

- Managing requirements is one of the riskiest parts of Software Engineering
- However, the solution is not to attempt to specify and stabilize all requirements from the beginning (Waterfall approach).
- Requirements changes during the development process
 - On average 25% of the requirements changes
- Therefore SW must be developed in an iterative way with frequent feedback and adjustment to changes.



Categories of Requirements

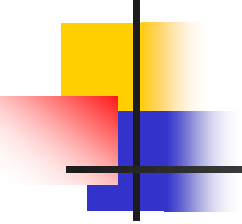
FURPS+ model :

- **Functionality**
- **Usability**
 - User interface, help, documentation
- **Reliability**
 - Frequency of failure, recoverability...
- **Performance**
 - Response times, resource usage...
- **Supportability**
 - Adaptability, maintainability, internationalization

- + in FURPS + (everything else...)
 - Design constraints
 - Implementation requirements
 - Interface requirements
 - Physical requirements

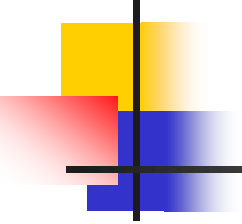


**Use as
Checklist**



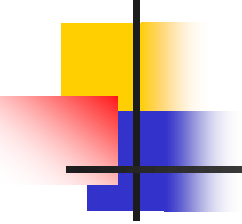
Requirements Artifacts

- **Vision document**
 - Summarizes high level requirements
 - Business case
 - Project big idea
- **Use-case model**
 - Functional requirements described as a set of typical scenarios of using the system
- **Supplementary specifications**
 - All other requirements (non-functional)

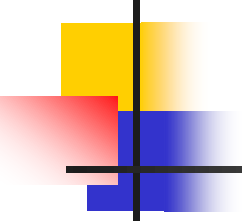


Alternative categorization

- **Functional requirements**
- **Non-functional requirements**

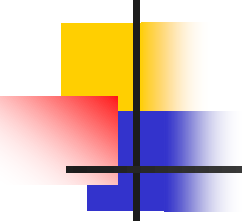


Use cases or User stories



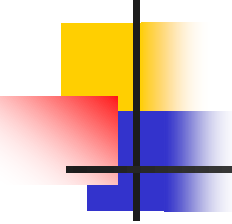
Use cases

- **What is a use case?**
- Before agile entered the tech scene to go on and replace the waterfall method as the most popular software development methodology, use cases were a thing. They were developed towards the end of the 1980s as a ULM (Unified Modelling Language) diagram.
- In some respects, use cases are similar to user stories, in that they also use plain language and are focused on achieving a particular goal for the user. They are also useful for both technical and non-technical stakeholders. However, unlike user stories, user flows focus more on the functionalities of a feature or process being developed, that is, how a system plays out. It focuses on the various steps taken, both by the user and the system to achieve a goal.



Use cases- what is it?

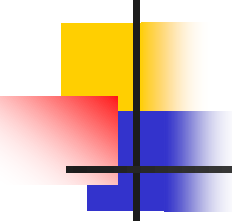
- Use cases are **text** stories, widely used to discover and record requirements; they are the **F** in FURPS+
- Use cases are not diagrams they are text : UML use case diagrams are trivial to learn; the real analysis skill is to identify and write good use cases.
- A use case diagram is useful as an overview of the functionalities of an application; it also defines the application's context (what is part of it and what isn't).



Definitions

- **Actor:** something with behaviour, such as a person, computer system, or organization, e.g. a cashier, a player.
- **Scenario:** specific sequence of actions and interactions between actors and the system. It is one particular story of using a system, or one path through a use case.
 - e.g. the scenario of successfully purchasing items with cash.
- **Use case:** a collection of related success and failure scenarios that describe actors using a system to support a goal.

A key point is to focus on the question “how can using the system provide observable value to the user, or fulfill their goals?”



Three kinds of actors

Actors are **roles** played by people, organisations, other software and devices that interact with the application.

Primary actors

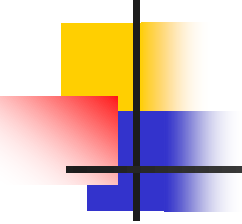
- Those that have user goals fulfilled through using services of the system

Supporting actors

- Provides a service to the system

Offstage actor

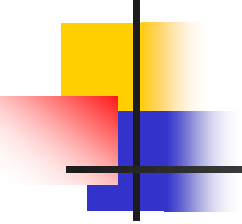
- has an interest in the behaviour of the use case, but is not primary or supporting. (e.g. tax agency)



Use case example

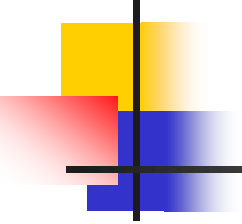
Process Sale

A customer arrives at a checkout with items to purchase. The cashier uses the POS system to record each purchased item. The system presents a running total and line-item details. The customer enters payment information, which the system validates and records. The system updates inventory. The customer receives a receipt from the system and then leaves with the items.



Use case types and formats

- Uses cases may be written in three formality types
 - **Brief Use Case** : one-paragraph summary, usually of the main success scenario. Created during early requirements analysis, to get a quick sense of subject and scope. May take only a few minutes to create; (e.g. Process sale)
 - **Casual Use Case** : Informal paragraph format. Multiple paragraphs that cover various scenarios. A refinement of a brief use case.
 - **Fully Dressed Use Case** : All steps and variations are written in detail, and there are supporting sections, such as preconditions and success guarantees.



Example of casual Use case

Handle return

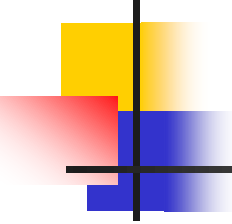
Main success scenario: A customer arrives at a checkout with items to return. The cashier uses the POS system to record each returned item...

Alternate scenarios:

If the customer paid by credit, and the reimbursement transaction to their credit card is rejected, inform the customer and pay them with cash.

If item identifier not found in the system, notify the Cashier and suggest manual entry of the identifier code.

...



Fully-dressed example: Process Sale

Use case UC1: Process Sale

Primary Actor: Cashier

Stakeholders and Interests:

- Cashier: Wants accurate and fast entry, no payment errors, ...
- Salesperson: Wants sales commissions updated.

...

Preconditions: Cashier is identified and authenticated.

Success Guarantee (Postconditions):

- Sale is saved. Tax correctly calculated.

...

Main success scenario (or basic flow): [see next slide]

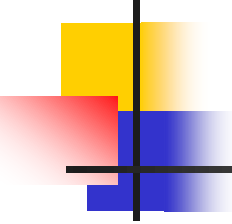
Extensions (or alternative flows): [see next slide]

Special requirements: Touch screen UI, ...

Technology and Data Variations List:

- Identifier entered by bar code scanner,...

Open issues: What are the tax law variations? ...



Fully dressed example: Process Sale (cont.)

Main success scenario (or basic flow):

The Customer arrives at a POS checkout with items to purchase.

The cashier records the identifier for each item. If there is more than one of the same item, the Cashier can enter the quantity as well.

The system determines the item price and adds the item information to the running sales transaction. The description and the price of the current item are presented.

On completion of item entry, the Cashier indicates to the POS system that item entry is complete.

The System calculates and presents the sale total.

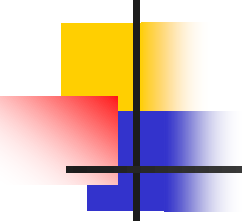
The Cashier tells the customer the total.

The Customer gives a cash payment ("cash tendered") possibly greater than the sale total.

Extensions (or alternative flows):

If invalid identifier entered. Indicate error.

If customer didn't have enough cash, cancel sales transaction.



Alternative notation

- A main alternative use case notation is to present the use case as a *conversation* between the actors and the system in a two column format:

Main Success Scenario:

Actor Action

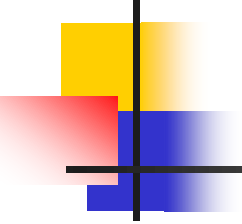
1. Customer arrives at a POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale
3. Cashier enters item identifier.

Cashier repeats steps 3-4 until indicates done.

6. Cashier tells Customer the total, and asks for payment.
7. Customer pays.

System Responsibility

4. Records each sale line item, presents item description and running total.
5. Presents total with taxes calculated.
8. Handles payment.
- 9...

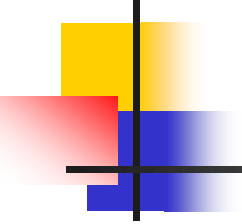


Goals and Scope of a Use Case

- At what level and scope should use cases be expressed?
- Focus on use cases at the level of **elementary business process** (EBP).
 - EBP: a task performed by one person in one place at one time which adds measurable business value and leaves the data in a consistent state.

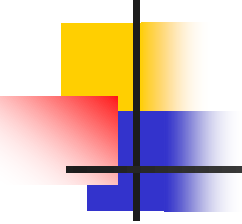
Are these valid use cases?

- Process Sale **YES**
 - Negotiate a supplier contract **NO**
 - Log in **NO**
 - Move Piece on Game Board **NO**
-
- A Use case consists normally of 5-10 steps. A simple small step is not a Use case.
 - It is usually useful to create separate “sub” use cases representing subtasks within a base use case.
 - e.g. Paying by credit

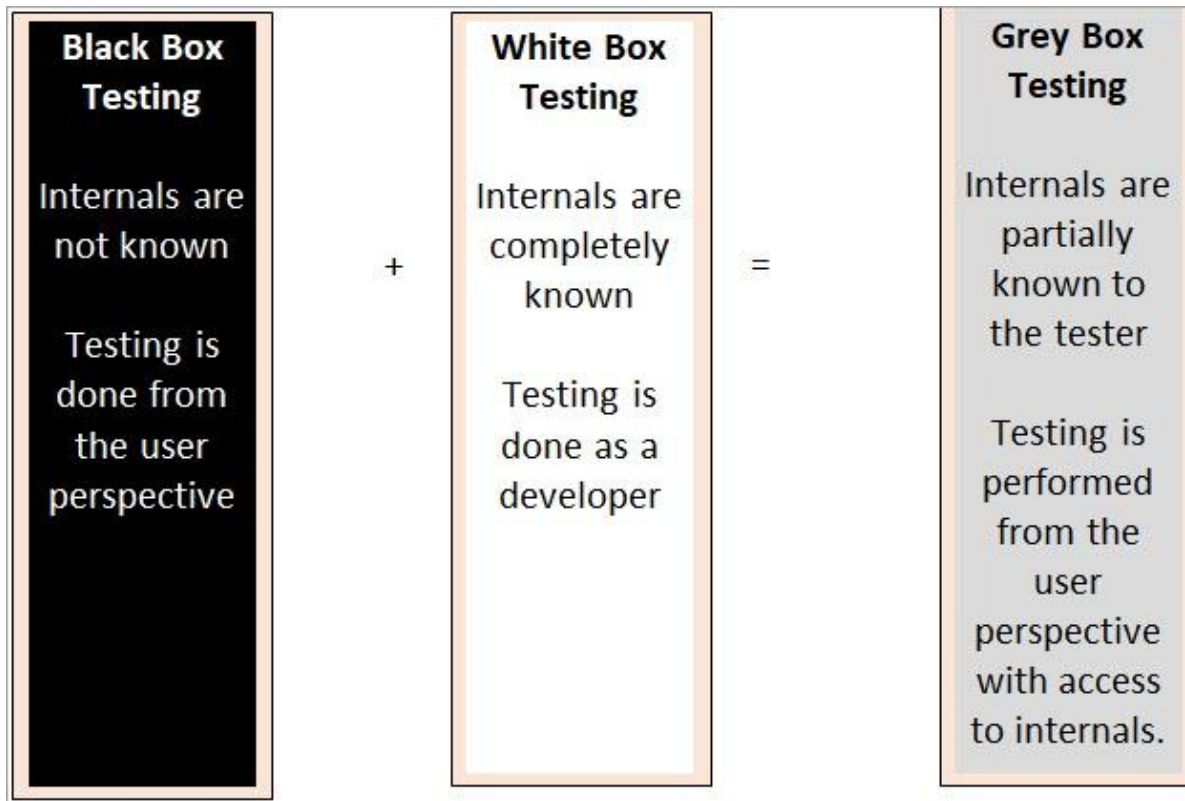


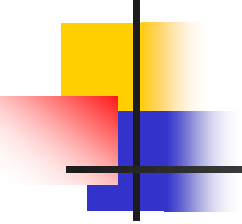
Guideline: Write Use Cases in UI-Free Style

- At this stage it is important to focus at the intent and keep the interface out. We are investigating the requirements it is too early for design decisions.
 - e.g. ~~"Admin enters ID and password in the dialog box"~~
- A UI-free use case:
 - Is simpler and shorter;
 - Allows you to focus on the goals of the actors rather than how they will be achieved at a UI-level;
 - Does not preclude the development of UI prototype to help eliciting the requirements and record them in use cases;
- If a strict constraint exists (for whatever reason) it must be recorded in the **Supplementary Specification** document
- Also try to write short (but complete!)



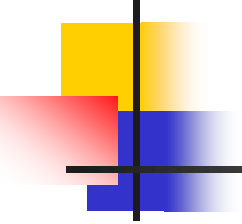
Types of testing





Guideline: Write Black-Box Use Cases

- Consider the system as a black-box at this stage. The use cases must specify **what** the system must do **not how** it will do it
 - Do write *'the system records the sale'* rather than *'The system writes the sale to the database via ...'* or even worse *'the system generates an SQL INSERT statement for the sale'*
- We must consider the uses cases from the actors point of view only: to stress observable user value and focus on users' typical goals.
- We must concentrate at this early stage on what the user need not on how it will work internally.

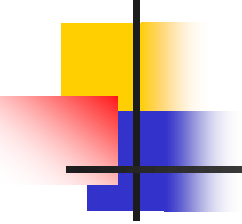


Finding primary actors, goals and use cases

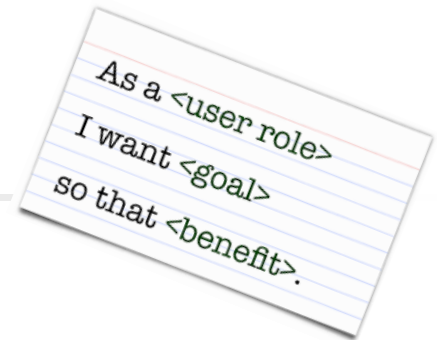
1. Choose the system boundary.
2. Identify primary actors.
3. For each actor identify their user goals.
4. Define use cases that satisfy user goals; name them according to their goal. Name must start with a verb

Exception to the rule one goal- one use case:

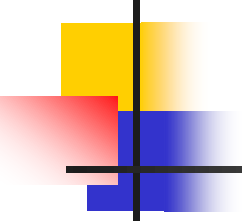
- CRUD (create, read, update, delete) one Use case
 - Manage X
-
- Of course, in iterative and evolutionary development, not all goals or use cases will be fully or correctly identified near the start. It's an evolving discovery.



User Stories



- **Drive the creation of the acceptance tests:**
 - There must be one or more tests to verify that a story has been properly implemented.
- **Different than Requirements:**
 - Should only provide enough detail to make a reasonably low risk estimate of how long the story will take to implement.
- **Different than Use Cases:**
 - Written by the Customer, not the Programmers, using the Customer's terminology



User Stories

- **What is a user story?**

- User stories define the who, the what and the why of a product feature. They're usually aimed at developers but are intended to be easily understood by everyone on the team, including clients. They outline what a user needs to get done, from that user's point of view.

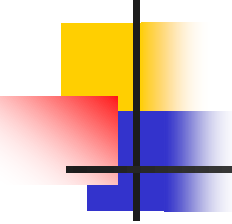
- **User story structure**

- They often comprise just one sentence or a couple of lines written in the following structure

- As a (user)
- I want to (perform some action)
- So I can (achieve some goal)

- An example of this could be:

- “As a user, I want to have a barcode ticket for my train pass on my phone so that I don't need to print it off and carry a paper around with me.”



User Story Examples

A user wants access to the system, so they find a system administrator, who enters in the user's First Name, Last Name, Middle Initial, E-Mail Address, Username (unique), and Phone Number.

Risk: Low

Cost: 2 points

The user must be able to search for a book.

Risk: High

Cost: (too large!)



The user must be able to search for a book by Title, and display the results as a list.

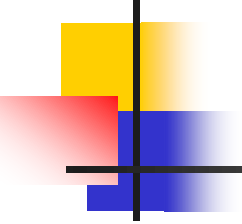
Risk: Med.

Cost: 1 point

The user must be able to search for a book by Category, and display the results as a list.

Risk: Med.

Cost: 2 points



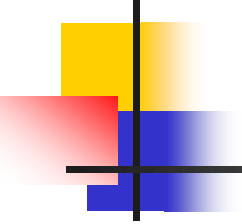
User stories template

User stories often follow a simple template:

***As a** <type of user>, **I want** <some goal> **so that** <some reason>*

Examples:

- **As a** user, **I want** to backup my entire hard drive, **so that** I can avoid losing valuable data.
- **As a** user, **I want** to indicate folders not to backup, **so that** my backup drive isn't filled up with things I don't need saved.
- **As a** power user, **I want** to specify files or folders to backup based on file size, date created and date modified, **so that**.....



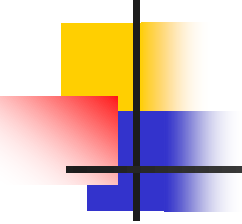
User stories vs use cases: differences

Degree of detail

- User stories are normally, and purposely, more vague. The user story provides a simplified, abridged description in layman's terms of what a feature should help the user do. This leaves it more open to interpretation and encourages more creativity and discussion on the part of the design and development teams.
- Use cases cover more ground by showing how the user should interact with the system and how the system should reciprocate. They go into further detail about how the individual steps in a feature's process.

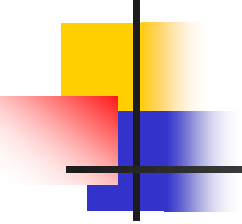
Sprint length

- The difference in detail is a big factor in user stories vs use cases. Perhaps one of the main differences between use cases and user stories is that the former are often completed over multiple sprints or iterations. On the other hand, user stories often contain features or details small enough to be completed over the course of one sprint. A team may even get through multiple user stories in one sprint.



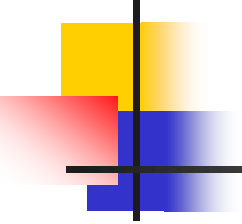
User stories vs use cases: differences

- User stories are much briefer, more succinct documents and generally tend to be used for smaller features. They can also be used to break larger features down into small steps or multiple user stories. Conversely, use cases tend to give more of an overview of a larger system functionality.
- Use cases generally list out the system functionalities required for each step of a feature's process. It shows you the input required from the user and the system.



Structure

- There's also a whopping big difference in the structure when it comes to user stories vs use cases. The user story is just a line or two of a statement about what exactly the user wants to achieve that can be written on a simple index card.
- This makes them quick and easy to generate. On the other hand, use cases can take a bit more time to put together, as they detail out each step and are often represented in a diagram.



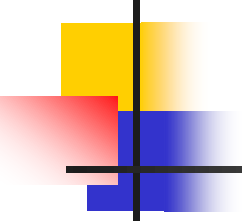
INVEST principle

- “I” ndependent (of all others)
- “N” egotiable (not a specific contract for features)
- “V” aluable
- “E” stimable (to a good approximation)
- “S” mall (so as to fit within an iteration)
- “T” estable (in principle, even if there isn’t a test for it yet)

- **When to use user stories or use cases**

- If you're caught between using user stories or use cases, the only answer is that it depends on your team and how big it is. Furthermore, it depends on the type of product you're working for, your organization and the client.
- The best rule of thumb when deciding between the two is to have a discussion with all stakeholders involved in the product.

Remember: to keep things agile, you should always try to avoid adding in too much detail



The takeaway – User stories vs use cases

- User stories and use cases share some similar concepts but are fundamentally different documents, each with their own unique benefits.
- User stories are faster to generate, spark conversation and offer a greater degree of creative flexibility.
- use cases offer up greater detail and a more structured approach to each step but take longer to generate

The takeaway – User stories vs use cases

